Kalasalingam Academy of Research and Education

Krishimitra : Farmer's Advisory chatbot

Project ID: PRJ079

Team ID: T079

Team GitHub Repository: github.com/psugreeva-ui/farmer-chatbot

Student Name	Registration Number	Individual GitHub Link
S Pavan Venkat Madhav Vinay	99230040775	github.com/psugreeva-ui/GenAI
P Venkata Harshavardhan Reddy	99230040739	github.com/Harsha36055/GenAI
Shaik Latheef	99230040764	github.com/latheefshaik123456789-lgtm/GENAI
Shaik Thanvir Ahmed	99230040768	github.com/thanvirahmedshaik/GEN-AI
T Anil Kumar	99230040781	github.com/anilyona1234/GenAI

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction ..... Section 1
2. Problem Statement & Objectives ..... Section 2
3. System Architecture ..... Section 3
4. Implementation Details & Tech Stack ..... Section 4
5. Results and Testing ..... Section 5
6. Conclusion & Future Scope ..... Section 6

1. Introduction

Agriculture supports a large share of India's population, yet most small and marginal farmers still lack timely access to expert advice. Extension officers are few, crop problems spread quickly, and most digital tools are written in English and need a fast internet connection. As a result, farmers often make decisions on spraying, selling and government support without reliable information.

KrishiMitra is a lightweight, mobile-optimized web application that acts as a farmer's advisory chatbot. It combines a Large Language Model (Google Gemini) with crop disease diagnosis from leaf photos, weather-based spray advice, mandi price and Minimum Support Price (MSP) comparison, and a catalog of government schemes. The interface works in six languages (English, Hindi, Punjabi, Marathi, Telugu and Tamil) and supports voice input and spoken replies, so that even farmers with limited reading ability can use it. When the network or the AI service is unavailable, an offline knowledge base keeps the core advice available.

2. Problem Statement & Objectives

Problem: Farmers need quick, local-language answers about crop diseases, weather risks, market rates and subsidies. Information is scattered across portals and manuals, is often available only in English, and is hard to use on low-end phones with slow 3G/4G connections.

Objectives:

- To develop a multi-turn conversational advisor that gives short, practical farming advice in the farmer's own language.
- To identify crop diseases from a photo of a leaf and return a structured diagnosis with organic and chemical remedies.
- To provide real-time weather data and a clear "spray / do not spray" recommendation.
- To show market prices against MSP and list central and state agricultural schemes.
- To remain usable on poor networks through a lightweight frontend and an offline fallback knowledge base.

3. System Architecture

KrishiMitra follows a simple client-server architecture. A framework-free web frontend talks to a Python FastAPI backend over REST endpoints. The backend calls external services (Gemini and Open-Meteo) and falls back to a local knowledge base whenever those calls fail.

3.1 Client Layer (Frontend)

The frontend is built with vanilla HTML5, CSS3 and JavaScript (ES6+), without heavy frameworks, so it loads quickly on rural networks. It provides tabbed navigation (Advice, Crop Doctor, Weather, Mandi, Schemes), a light/dark mode, and a language selector driven by a translations dictionary. The Web Speech API (SpeechRecognition) captures voice questions, and the Speech Synthesis API reads answers aloud. Leaf photos can be taken with the phone camera or dragged and dropped, and are resized in the browser before upload to save data.

3.2 Server Layer and Request Flow

When a farmer sends a question, the frontend posts the message, the recent conversation history and the selected language to **/api/chat**. The backend builds the conversation, adds a system prompt that restricts the model to short, safe, practical agricultural advice in the chosen language, and calls the Gemini model through the google-genai SDK. For photos, **/api/diagnose** sends the image to the same model with instructions to return a JSON diagnosis (crop, disease, confidence, severity, symptoms, remedies and prevention). The weather endpoint requests current conditions and a 5-day forecast from Open-Meteo and applies rule-based thresholds on wind, rain, temperature and humidity to decide whether spraying is safe.

3.3 Offline Fallback Layer

The module **knowledge_base.py** stores fallback disease remedies, MSP rates, indicative mandi prices, scheme details and keyword-based advice. If the API key is missing, the network fails or the AI service returns an error, the backend answers from this knowledge base and marks the response as "offline", so the farmer always receives a useful reply.

4. Implementation Details & Tech Stack

Component	Technology Used	Purpose
Programming Language	Python 3.10+	Core backend logic
Web Framework	FastAPI with Uvicorn	REST API and ASGI server
Large Language Model	Google Gemini (google-genai SDK)	Chat advice and multimodal leaf diagnosis
Weather Data	Open-Meteo API via httpx	Current weather and 5-day forecast
Offline Database	knowledge_base.py	Fallback remedies, MSP rates and schemes
Frontend	HTML5, CSS3, JavaScript (ES6+)	Lightweight, responsive mobile interface
Browser APIs	Web Speech, Speech Synthesis, Camera Capture, Drag & Drop	Voice input and output, photo capture

4.1 REST Endpoints

Endpoint	Method	Purpose
/api/chat	POST	Multi-turn agricultural advisor
/api/diagnose	POST	Leaf image upload and structured disease diagnosis
/api/weather	GET	Weather, 5-day forecast and spray safety
/api/mandi	GET	Market prices compared with MSP
/api/schemes	GET	Central and state scheme catalog
/api/health, /api/set-key	GET, POST	System status and runtime API key setup

4.2 Sample Model Configuration

To keep answers short and consistent, the chat endpoint sends a system instruction with a low temperature and a limited output length:

```
resp = await client.aio.models.generate_content( model=MODEL, contents=contents,
config=types.GenerateContentConfig(
system_instruction=SYSTEM_PROMPT.format(language=language),
temperature=0.4, max_output_tokens=600 ) )
```

5. Results and Testing

The application was tested feature by feature through its REST endpoints and the browser interface. The main test scenarios are summarised below.

Feature	Test Scenario	Expected Behaviour
Chat advisor	Ask a crop question in each of the six languages	Short reply in the selected language
Voice	Speak a question and play the reply aloud	Text is filled by speech and read back
Crop Doctor	Upload or capture a leaf photo	Structured diagnosis with remedies
Weather	Open the Weather tab with location access	Conditions, forecast and spray advice
Offline mode	Remove the API key or disconnect the network	Reply from the knowledge base marked offline
Mandi and Schemes	Open both tabs	MSP comparison and scheme list displayed

6. Conclusion & Future Scope

Conclusion: KrishiMitra brings advisory, disease diagnosis, weather guidance, market rates and scheme information into one lightweight, multilingual web application. By combining a generative AI model with an offline knowledge base and voice support, it gives rural farmers practical help even on slow or unreliable networks.

Future Scope: Future versions will connect to live AGMARKNET mandi prices, add state-specific scheme details, and provide a service worker so the app opens without internet. We also plan to ground answers in agricultural extension documents using retrieval, add user authentication, translate the offline fallback into all supported languages, and collect farmer feedback to keep improving the advice.